ESTRUCTURAS DE CONTROL

REPETICIÓN

XIOMARA SALVATIERRA BALBÍN

LA CLASE PASADA...

- Operadores relacionales
- Operadores lógicos
- IF (condicional simple)
- IF – ELSE (condicional doble)
- Condicionales anidadas
- Selección múltiple (switch)
- Diagramas de flujo
- Condiciones compuestas

¿POR QUÉ NECESITAMOS CICLOS?

- Automatizan tareas repetitivas.
- Ahorran código.
- Permiten trabajar con grandes cantidades de datos.
- Son base para estructuras más complejas (matrices, algoritmos, etc.).

```python
print("Hola")
print("Hola")
print("Hola")
print("Hola")
print("Hola")
```

```python
for i in range(5):
    print("Hola")
```

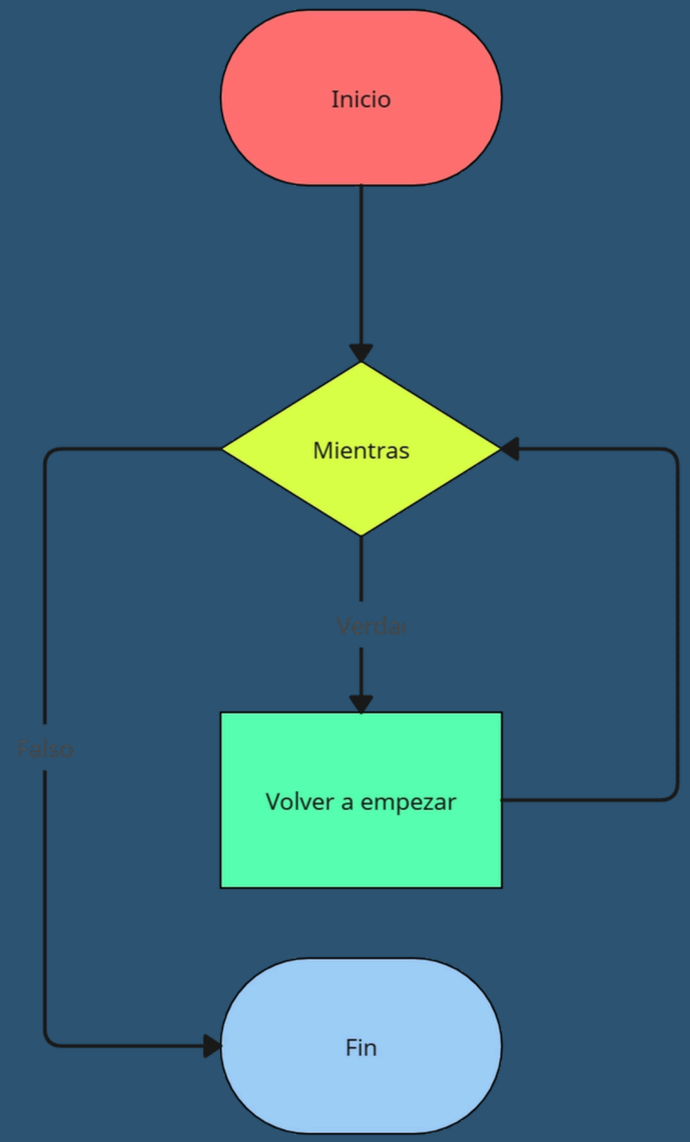
¿CÓMO FUNCIONA UN CICLO?

TODO CICLO NECESITA:

- UNA CONDICIÓN
- ALGO QUE CAMBIE EN CADA REPETICIÓN
- UN PUNTO DONDE TERMINE

SI NO HAY ALGO QUE CAMBIE → CICLO INFINITO

¿POR QUÉ NECESITAMOS CICLOS?



En un diagrama de flujo:

- Inicio
- Evaluar condición
- ¿Se cumple?

Sí → Ejecutar instrucciones → volver a condición

No → Termina

Si tengo que pedir una contraseña hasta que sea correcta…
¿Eso es decisión o repetición?

CICLO MIENTRAS (WHILE)

Ejecuta un conjunto de instrucciones mientras una condición sea verdadera.

Estructura en pseudocódigo

```
INICIO
    contador = 1
    MIENTRAS contador <= 5 HACER
        ESCRIBIR contador
        contador = contador + 1
    FIN MIENTRAS
FIN
```

6

1 2 3 4 5

¿QUÉ OCURRE PASO A PASO?

- Se inicializa el contador.
- Se evalúa la condición.
- Si es verdadera → ejecuta.
- Se actualiza la variable.
- Vuelve a evaluar.
- Cuando es falsa → termina

El ciclo **MIENTRAS** es un ciclo de **entrada controlada.**

Eso significa:
Puede que nunca se ejecute si la condición es falsa desde el inicio.

```
contador = 10

MIENTRAS contador = 5
HACER
    ESCRIBIR contador
FIN MIENTRAS
```

CICLO HACER - MIENTRAS (DO - WHILE)

Ejecuta primero las instrucciones y luego evalúa la condición.
A diferencia del WHILE...
WHILE → primero pregunta, luego ejecuta
DO–WHILE → primero ejecuta, luego pregunta

Estructura en pseudocódigo

```
INICIO
    contador = 1
    HACER
        ESCRIBIR contador
        contador = contador + 1
    MIENTRAS contador <= 5
FIN
```

6

1 2 3 4 5

¿QUÉ OCURRE PASO A PASO?

- Ejecuta las instrucciones.
- Luego evalúa la condición.
- Si es verdadera → repite.
- Si es falsa → termina.

El **DO-WHILE** es un ciclo de **salida controlada.**

Con WHILE:

```
MIENTRAS contador < 5 HACER
  ESCRIBIR contador
FIN MIENTRAS
```

No se ejecuta nunca.

Con HACER–MIENTRAS:

```
HACER
  ESCRIBIR contador
MIENTRAS contador < 5
```

Se ejecuta una vez.

CICLO HACER - MIENTRAS (DO - WHILE)

EJERCICIO 1: Mostrar números del 1 al 5

Con MIENTRAS

```
INICIO
   contador = 1

   MIENTRAS contador <= 5 HACER
      ESCRIBIR contador
      contador = contador + 1
   FINMIENTRAS
FIN
```

Con HACER-MIENTRAS

```
INICIO
   contador = 1

   HACER
      ESCRIBIR contador
      contador = contador + 1
   MIENTRAS contador <= 5
FIN
```

EJERCICIO 2: Pedir número hasta que sea positivo

Con MIENTRAS

```
INICIO
   LEER numero

   MIENTRAS numero <= 0 HACER
         ESCRIBIR "Ingrese un número
positivo:"
      LEER numero
   FINMIENTRAS

   ESCRIBIR "Número válido"
FIN
```

Con HACER-MIENTRAS

```
INICIO
   HACER
         ESCRIBIR "Ingrese un número
positivo:"
      LEER numero
   MIENTRAS numero <= 0

   ESCRIBIR "Número válido"
FIN
```

Actividad:

¿Cuál estructura necesita una lectura previa?
WHILE

¿Cuál garantiza al menos una ejecución?
DO WHILE

¿En qué casos usarían cada uno?
WHILE: Cuando no sabemos si debe ejecutarse
DO WHILE: Al menos una vez

¿Qué pasa si la condición es falsa desde el inicio?
WHILE: No se ejecuta
DO WHILE: Se ejecuta una vez

CICLO PARA (FOR)

- La repetición está basada en un rango o cantidad conocida.
- El contador es parte natural del problema.

Estructura en pseudocódigo

Forma general:

```
PARA variable → valor_inicial HASTA
valor_final HACER
   instrucciones
FIN PARA
```

Ejemplo:

i = 1, 2, 3, 4, 5

```
PARA i → 1 HASTA 5 HACER
   ESCRIBIR i
FIN PARA
```

1 2 3 4 5

¿Qué ocurre paso a paso?

El FOR integra tres cosas en una sola línea:

1. Inicialización
2. Condición
3. Actualización automática

Eso lo hace más compacto que el WHILE.

PASO A PASO

1. $i \leftarrow 1$
2. ¿$i \leq 5$?
3. Ejecuta
4. $i \leftarrow i + 1$

Vuelve a evaluar...

CICLO PARA (FOR)

- La repetición está basada en un rango o cantidad conocida.
- El contador es parte natural del problema.

Mostrar números del 1 al 10

```
PARA i ← 1 HASTA 10 HACER
  ESCRIBIR i
FIN PARA
```

Sumar los primeros 5 números

```
suma ← 0
PARA i ← 1 HASTA 5 HACER
  suma ← suma + i
FIN PARA
ESCRIBIR suma
```

Mostrar números pares del 2 al 20
Opción 1 (incremento de 2 en 2):

```
PARA i ← 2 HASTA 20 CON PASO 2
 HACER
  ESCRIBIR i
FIN PARA
```

Tabla de multiplicar de un número

```
LEER numero
PARA i ← 1 HASTA 10 HACER
  ESCRIBIR numero, " x ", i, " = ", numero * i
FIN PARA
```

7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
.
.
.
7 x 10 = 70

CICLOS ANIDADOS

Un ciclo anidado es:
Un ciclo dentro de otro ciclo.
Es decir, mientras el ciclo externo se ejecuta, el interno se ejecuta completamente en cada repetición.

Estructura en pseudocódigo

```
PARA i ← 1 HASTA 3 HACER
    PARA j ← 1 HASTA 2 HACER
        ESCRIBIR i, j
    FIN PARA
FIN PARA
```

¿QUÉ OCURRE PASO A PASO?

Cuando i = 1
→ j = 1
→ j = 2
Cuando i = 2
→ j = 1
→ j = 2
Cuando i = 3
→ j = 1
→ j = 2

Ejemplo clásico: Tabla de multiplicar

```
PARA i ← 1 HASTA 3 HACER
  PARA j ← 1 HASTA 3 HACER
    ESCRIBIR i * j
  FIN PARA
FIN PARA
```

i = 1 2 3

j = 1 2 3

3 → 1, 2, 3

ESCRIBIR

1 2 3 2 4 6 3 6 9

BREAK

Termina el ciclo inmediatamente, sin importar la condición. Es una salida forzada.

Estructura en pseudocódigo

```
PARA i ← 1 HASTA 10 HACER
    SI i = 5 ENTONCES
        BREAK
    FIN SI
    ESCRIBIR i
FIN PARA
```

¿Qué pasa?
Se imprime:
1 2 3 4
Cuando i = 5 → se rompe el ciclo.

CONTINUE

Salta la iteración actual y pasa a la siguiente. No termina el ciclo. Solo omite esa vuelta.

Estructura en pseudocódigo

```
PARA i ← 1 HASTA 5 HACER
    SI i = 3 ENTONCES
        CONTINUE
    FIN SI
    ESCRIBIR i
FIN PARA
```

¿Qué pasa?
Resultado:
1 2 4 5
El 3 no se imprime, pero el ciclo sigue.

CONTADOR

Aumenta (o disminuye) en una cantidad fija cada vez que ocurre algo.
Normalmente aumenta de 1 en 1.

Estructura en pseudocódigo

```
contador ← 0

PARA i ← 1 HASTA 5 HACER
   contador ← contador + 1
FIN PARA
```

Contando repeticiones.

ACUMULADOR

Un acumulador es una variable que:
Guarda una suma progresiva de valores.
No suma 1.
Suma valores variables.

Estructura en pseudocódigo

```
total ← 0

PARA i ← 1 HASTA 3 HACER
   LEER numero
   total ← total + numero
FIN PARA

ESCRIBIR total
```

Acumulando valores.

CONTADOR

Aumenta (o disminuye) en una cantidad fija cada vez que ocurre algo.
Normalmente aumenta de 1 en 1.

ACUMULADOR

Un acumulador es una variable que:
Guarda una suma progresiva de valores.
No suma 1.
Suma valores variables.

Calcular el promedio de N números.

```
suma = 0
contador = 0

PARA i ← 1 HASTA N HACER
   LEER numero
   suma ← suma + numero
   contador ← contador + 1
FIN PARA

promedio ← suma / contador
ESCRIBIR promedio
```

- ***suma → acumulador***
- ***contador → contador***

✓ **Contadores**
Si aún no has contado nada:
contador ← 0
Porque al inicio:
Han ocurrido 0 eventos.

✓ **Acumuladores**
Si aún no has sumado nada:
suma ← 0
Porque el total inicial es 0.

Si representa el "primer elemento", empieza en 1.

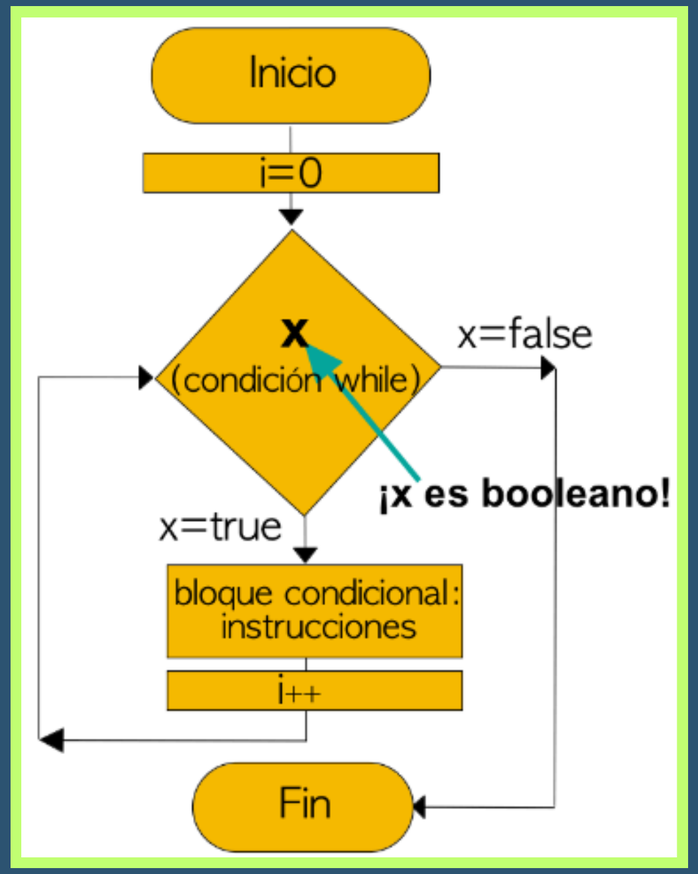
✓ **Numeración de repeticiones**
PARA i ← 1 HASTA 5 HACER

Porque queremos:
1, 2, 3, 4, 5
No tiene sentido empezar en 0 si estamos contando personas, días, posiciones naturales.

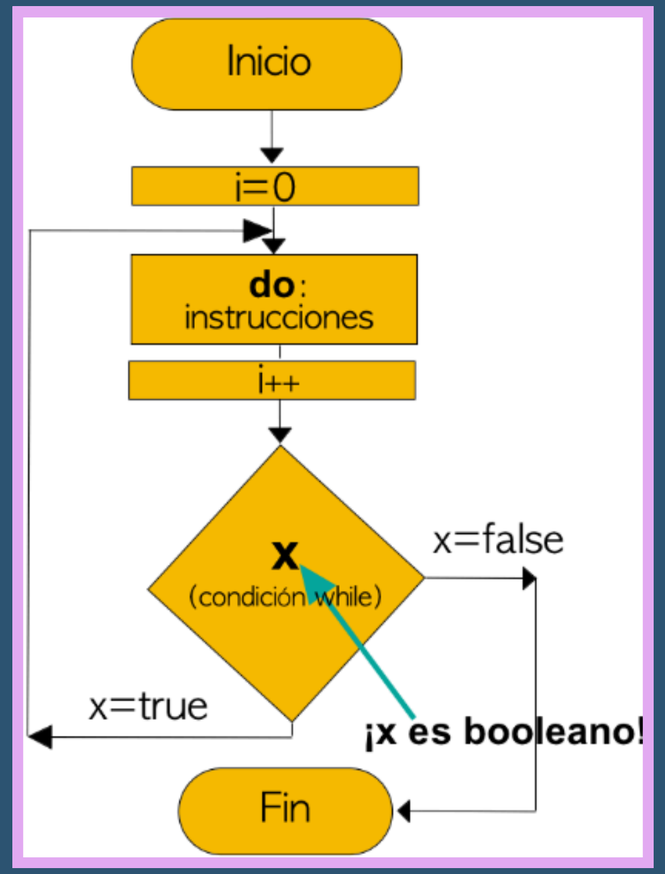
✓ **Tablas de multiplicar**
Empiezan en 1.

DIAGRAMAS DE FLUJO PARA CICLOS

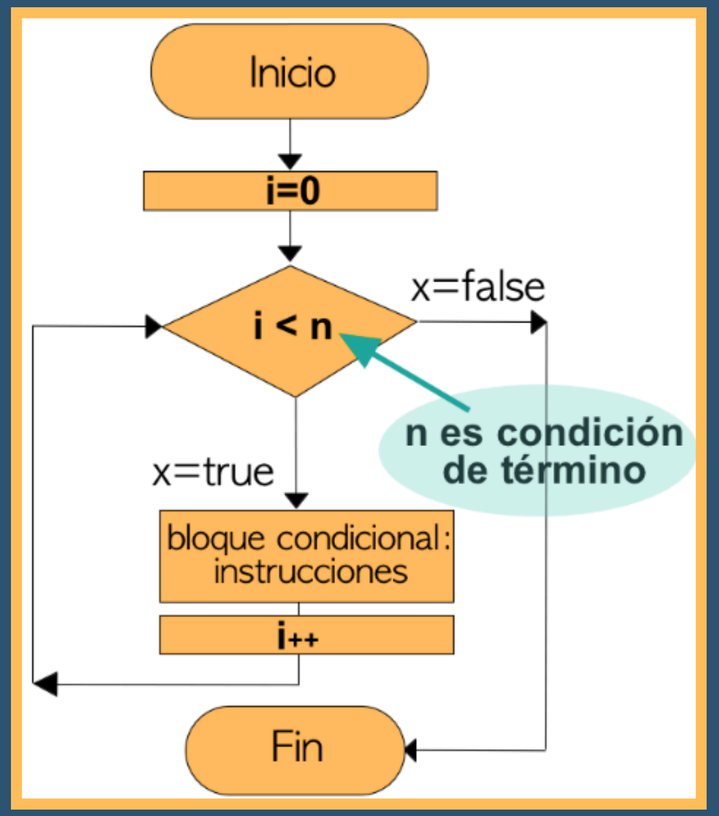
WHILE



DO WHILE

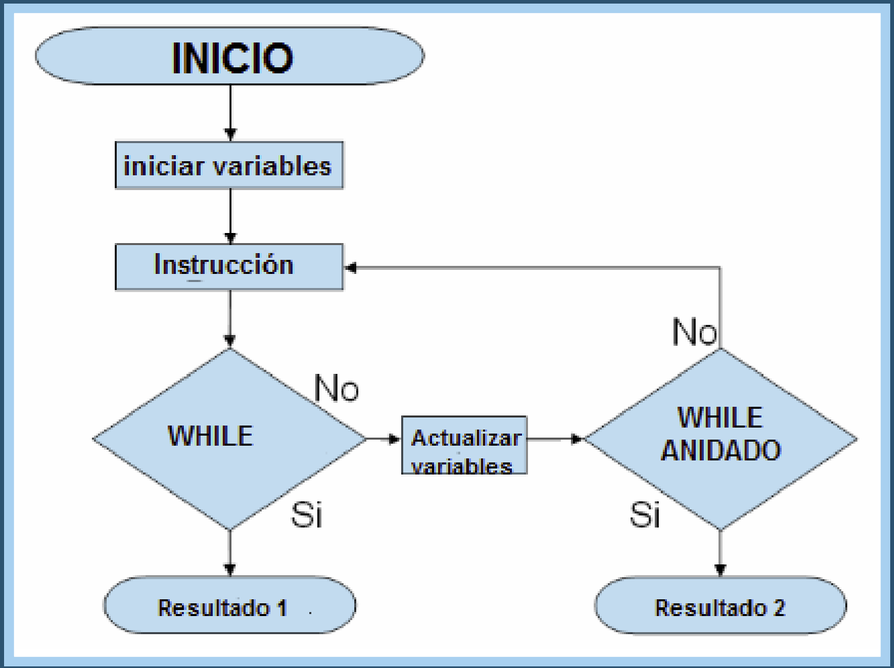


FOR

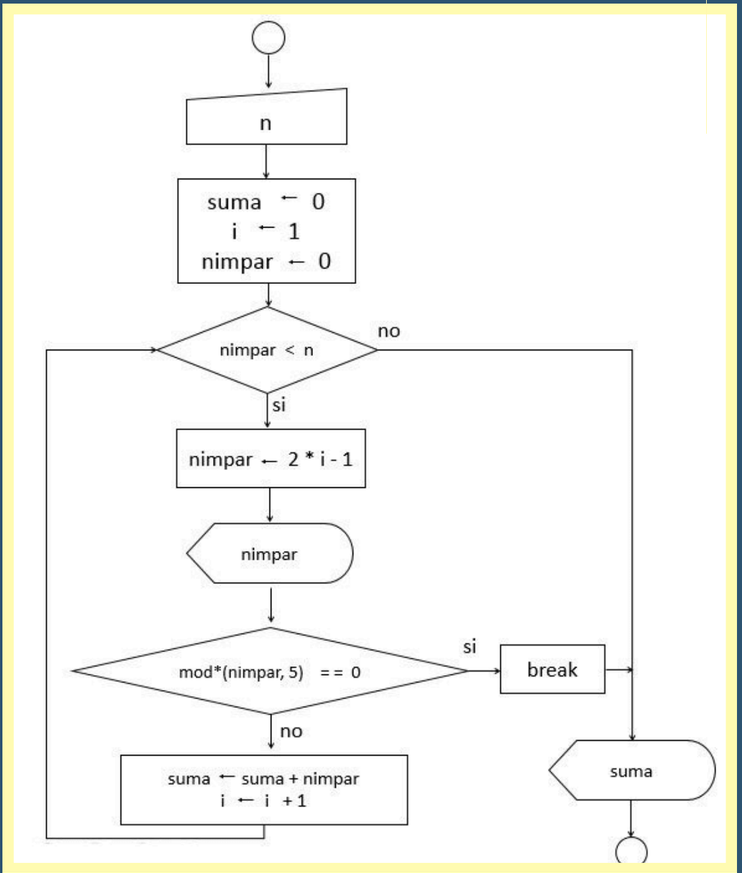


DIAGRAMAS DE FLUJO PARA CICLOS

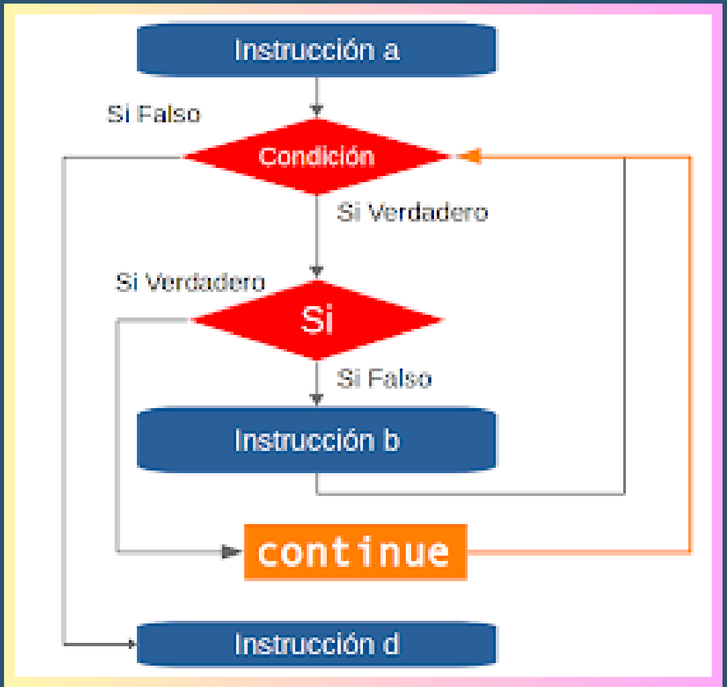
CICLOS ANIDADOS



BREAK



CONTINUE



RESUMEN FINAL

- ✓ **WHILE** → Repite mientras la condición sea verdadera
- ✓ **DO-WHILE** → Ejecuta al menos una vez
- ✓ **FOR** → Repetición con número definido
- ✓ **Ciclos anidados** → Un ciclo dentro de otro
- ✓ **BREAK** → Sale del ciclo
- ✓ **CONTINUE** → Salta una repetición
- ✓ **Contador** → Cuenta eventos
- ✓ **Acumulador** → Suma valores
- ✓ **Diagramas** → Representación visual del flujo

PROYECTO DE FIN DE CURSO - 1RA ENTREGA

MINI TORNEO DE VIDEOJUEGOS

INDICACIONES

Se desea desarrollar un algoritmo en pseudocódigo que simule un mini torneo de videojuegos.

El programa deberá:

1. Registrar 5 jugadores.
2. Para cada jugador, ingresar 3 puntajes correspondientes a 3 rondas.
3. Calcular el puntaje total de cada jugador.
4. Mostrar en pantalla el nombre del jugador y su puntaje total.

Condiciones:

- Los puntajes no pueden ser negativos.
- Si se ingresa un puntaje negativo, se debe mostrar un mensaje y tomar el valor como 0.
- Debe utilizar ciclos para repetir jugadores y rondas.
- No se permite el uso de funciones ni arreglos.

Aplicar correctamente:

- Variables
- Condicionales
- Ciclos anidados
- Acumuladores

```
INICIO

    DEFINIR nombre COMO TEXTO
    DEFINIR nota COMO ENTERO
    DEFINIR sumaNotas COMO ENTERO
    DEFINIR promedio COMO REAL

    PARA i DESDE 1 HASTA 5 HACER

        ESCRIBIR "Ingrese nombre del estudiante:"
        LEER nombre

        sumaNotas = 0

        PARA j DESDE 1 HASTA 3 HACER

            ESCRIBIR "Ingrese nota ", j
            LEER nota

            SI nota < 0 O nota > 100 ENTONCES
                ESCRIBIR "Nota inválida. Se tomará como 0."
                nota = 0
            FIN SI

            sumaNotas = sumaNotas + nota

        FIN PARA

        promedio = sumaNotas / 3

        ESCRIBIR "Promedio de ", nombre, " es: ", promedio

        SI promedio >= 60 ENTONCES
            ESCRIBIR "Estado: APRUEBA"
        SINO
            ESCRIBIR "Estado: REPRUEBA"
        FIN SI

        ESCRIBIR "--------------------------------"

    FIN PARA

FIN
```

EJEMPLO DE PROYECTO "CONTROL DE NOTAS DE UN CURSO"

- Registrar 5 estudiantes.
- Para cada estudiante, ingresar 3 calificaciones.
- Calcular el promedio de cada estudiante.
- Mostrar el nombre del estudiante y su promedio.
- Indicar si el estudiante APRUEBA o REPRUEBA.

```
INICIO

    DEFINIR nombre COMO TEXTO
    DEFINIR nota COMO ENTERO
    DEFINIR sumaNotas COMO ENTERO
    DEFINIR promedio COMO REAL

    PARA i DESDE 1 HASTA 5 HACER

        ESCRIBIR "Ingrese nombre del estudiante:"
        LEER nombre

        sumaNotas = 0

        PARA j DESDE 1 HASTA 3 HACER

            ESCRIBIR "Ingrese nota ", j
            LEER nota

            SI nota < 0 O nota > 100 ENTONCES
                ESCRIBIR "Nota inválida. Se tomará como 0."
                nota = 0
            FIN SI

            sumaNotas = sumaNotas + nota

        FIN PARA

        promedio = sumaNotas / 3

        ESCRIBIR "Promedio de ", nombre, " es: ", promedio

        SI promedio >= 60 ENTONCES
            ESCRIBIR "Estado: APRUEBA"
        SINO
            ESCRIBIR "Estado: REPRUEBA"
        FIN SI

        ESCRIBIR "--------------------------------"

    FIN PARA

FIN
```

INPUT

OUTPUT

EJEMPLO DE PROYECTO "CONTROL DE NOTAS DE UN CURSO"

- Registrar 5 estudiantes.
- Para cada estudiante, ingresar 3 calificaciones.
- Calcular el promedio de cada estudiante.
- Mostrar el nombre del estudiante y su promedio.
- Indicar si el estudiante APRUEBA o REPRUEBA.

¡GRACIAS!
NOS VEMOS LA PRÓXIMA CLASE